CreditFlow AI

Application Understanding Guide

Project: Multi-Agent Credit Underwriting & Risk Intelligence Platform

Purpose: Help authorized human underwriters analyze commercial credit applications using deterministic calculations, evidence retrieval, validation, and explainable AI recommendations.

> **Important:** CreditFlow does not autonomously approve or deny loans. Every recommendation requires an authorized human decision. All included examples and policies are synthetic.

1. Start with the running application

- Dashboard: `http://127.0.0.1:5173`
- API: `http://127.0.0.1:8000`
- OpenAPI documentation: `http://127.0.0.1:8000/docs`

2. Business process

1. Create a commercial credit case.
2. Upload financial and collateral documents.
3. Extract financial facts with source references.
4. Calculate ratios using deterministic Decimal functions.
5. Retrieve effective institutional policies.
6. Apply versioned risk rules.
7. Verify calculations, claims, and citations.
8. Generate an underwriting recommendation and report.
9. Pause the workflow for an authorized human reviewer.
10. Record the reviewer action and append an integrity-protected audit event.

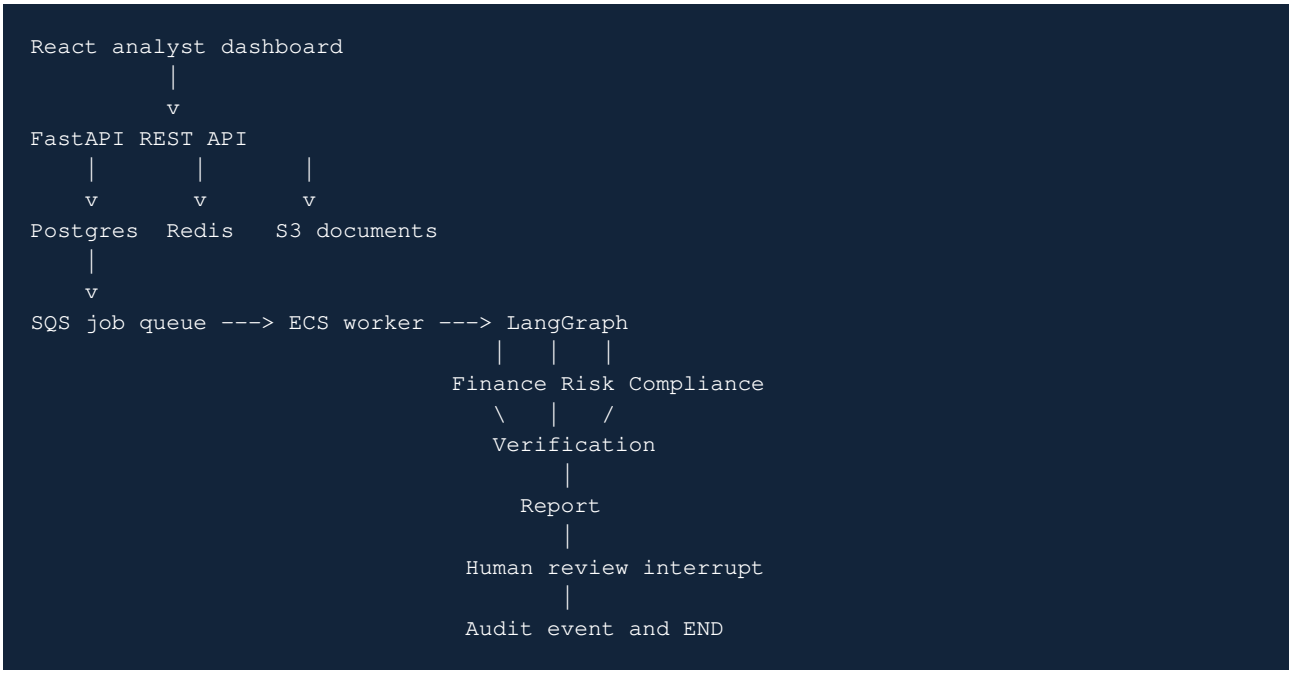
3. Demonstration cases

Case	Input	Expected behavior
A — Stable	`case_a_financials.csv`	Low risk; proceed to human review with supporting evidence.
B — Leveraged	`case_b_financials.csv`	High risk; low coverage, leverage, liquidity, or collateral warnings.

C — Incomplete	`case_c_incomplete.csv`	Insufficient data; request missing information and escalate.

Create a case in the dashboard, upload one of the files from `sample_data/documents/`, start analysis, inspect metrics and citations, submit a review, and open the audit timeline.

4. End-to-end architecture



5. Follow one request through the code

API entry point

`src/creditflow/main.py` creates FastAPI, middleware, tracing, routes, health checks, metrics, and standardized errors.

REST endpoints

`src/creditflow/api/v1/routes.py` contains case creation, upload, asynchronous analysis, job status, analysis, reports, human review, audit history, and policy search.

Local application service

`src/creditflow/services/runtime.py` connects the credential-free local demonstration. It creates cases, extracts CSV facts, launches LangGraph in the background, records reports, resumes human review, and appends audit events.

6. LangGraph workflow

Read `workflows/state.py`, `workflows/graph.py`, and `docs/langgraph.md`.

```
Load application -> Validate documents -> Extract facts
      -> Validate extraction -> Retrieve policies
      -> Financial analysis
           +-> Risk analysis ---------+
           +-> Compliance analysis --+-> Verification
                                     | valid
                                     v
                                Generate report
                                     |
                               Human interrupt
                                     |
                           Command(resume=review)
                                     |
                                Audit and END
```

Verification uses bounded correction attempts. Unsupported citations are removed rather than fabricated. An unresolved failure produces an insufficient-data recommendation instead of silently continuing.

7. Financial correctness

`risk/calculations.py` owns authoritative arithmetic. Values use `Decimal` and documented rounding. Every metric contains its formula, input values, source IDs, result, validation status, and missing inputs.

Examples include DSCR, current ratio, debt-to-equity, loan-to-value, and operating margin. The LLM may explain these results but cannot replace their deterministic calculation.

8. Document ingestion

`ingestion/extractors.py` validates filename, MIME type, magic bytes, and size; calculates SHA-256 checksums; extracts PDF or CSV content; reports OCR requirements; identifies document types; and produces page-preserving chunks with immutable citation IDs.

9. Hybrid retrieval and citations

`retrieval/hybrid.py` combines semantic and keyword results with Reciprocal Rank Fusion, reranks a candidate set, builds bounded grounded context, and validates source IDs. PostgreSQL migrations provide pgvector, HNSW, full-text GIN, policy metadata indexes, embedding versioning, and tenant row-level security.

10. LLM integration

`llm/providers/openai_responses.py` uses the OpenAI Responses API with structured Pydantic outputs. It handles incomplete responses, refusals, malformed output, and provider failures. The deprecated Assistants API is not used.

`llm/providers/mock.py` [illegible] `llm/routing.py` [illegible] fallback only to providers with approved structured-output, function-calling, and sensitive-data capabilities.

11. Security and auditability

- OIDC signature, issuer, audience, expiry, subject, JWKS, tenant, and role validation.
- Tenant identity comes from verified authentication, not request payloads.
- PostgreSQL row-level security provides defense in depth.
- Uploaded and retrieved text is untrusted and cannot authorize tools.
- Tools are allowlisted; the LLM cannot execute arbitrary SQL or code.
- Case versions prevent stale human review actions.
- Audit events are append-only and SHA-256 hash chained.

12. Recommended reading order

1. `README.md`
2. `docs/architecture.md`
3. `docs/langgraph.md`
4. `docs/retrieval.md`
5. `docs/database.md`
6. `docs/security.md`
7. `docs/interview-guide.md`
8. `PROGRESS.md`

13. Verification commands

```
uv run ruff format --check .
uv run ruff check .
uv run mypy src
uv run pytest --cov=creditflow
uv run python evals/run_evaluation.py
uv run pip-audit
npm --prefix web run build
```

14. Current readiness boundary

The local mock-mode vertical slice is executable. Production deployment still requires complete PostgreSQL/S3/SQS composition wiring, live identity and provider validation, Docker and AWS staging tests, representative load testing, and institutional security, privacy, fair-lending, legal, and model-risk approvals.

certification or achieved enterprise-performance claim.
